

Scala For DS

Name : DHRUV GAVANKAR	Roll no.: S007
Class : SYIT	Assignment Date : 22/09/2026

Practical no.27

27) Implement one-hot encoding for a categorical column in a dataset.

INPUT:

```
object Practical extends App
{
  val data = List("Red", "Blue", "Green", "Red", "Blue")
  for (value <- data)
  {
    val r = if (value == "Red") 1 else 0
    val b = if (value == "Blue") 1 else 0
    val g = if (value == "Green") 1 else 0
    println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
  }
}
```

OUTPUT:

```
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0

Process finished with exit code 0
```

Practical no.28

28) Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

INPUT:

```
import breeze.linalg._

object Practical extends App
{
  val A = DenseVector(2.0, 3.0)
  val B = DenseVector(5.0, 7.0)

  val distance = norm(A - B)

  println("Euclidean Distance: " + distance)

  val C = DenseVector(2.5, 3.5)

  val d1 = norm(C - A)
  val d2 = norm(C - B)

  if (d1 < d2)
  |   println("Nearest Neighbor: A")
  else
  |   println("Nearest Neighbor: B")
}
```

OUTPUT:

```
"C:\Program Files\Java\jdk1.8.0_341\bin\java.exe" ...
Euclidean Distance: 5.0
Nearest Neighbor: A

Process finished with exit code 0
```

Practical no.29

29) Create a simple Spark MLlib pipeline to classify data. Use logistic regression or decision trees.

INPUT:

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.ml.classification.LogisticRegression

object Practical29 extends App {

  val spark = SparkSession.builder()
    .appName("Classification")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val data = Seq(
    (1.0, 0.0),
    (2.0, 0.0),
    (3.0, 0.0),
    (7.0, 1.0),
    (8.0, 1.0),
    (9.0, 1.0)
  ).toDF("marks", "label")

  val a = new VectorAssembler()
    .setInputCols(Array("marks"))
    .setOutputCol("features")
```

```

val d = a.transform(data)

val model = new LogisticRegression()
    .setLabelCol("label")
    .setFeaturesCol("features")

val result = model.fit(d)

val prediction = result.transform(d)

prediction.select("marks", "label", "prediction").show()

spark.stop()
}

```

OUTPUT:

```

+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|        0.0|
|  2.0|  0.0|        0.0|
|  3.0|  0.0|        0.0|
|  7.0|  1.0|        1.0|
|  8.0|  1.0|        1.0|
|  9.0|  1.0|        1.0|
+-----+-----+-----+

```